Liu Jin — speaker script

SWE5006 Practice Project presentation, Team 8 BlocBuy, Monday 26 October 2026, 16:30–17:10 SGT (Zoom). My part: the full-form design problem and the schema (§2), then all of §3, testing. Source: branch docs/swe5006-markers, based on dev 0b3fe975. Speaking time is estimated at 130 words a minute. Markers (FILL, VERIFY, CONFIRM) must be gone after the 25 Oct rehearsal.

Run of show

Slide	Clock	Slot	Est.	Title	Rubric
22	16:37:50	30 s	30 s	Design problem: listing rules that depend on its state	2.5, G4
23	16:38:20	25 s	25 s	After: the State pattern, and why the team uses a table	2.5, 2.4, G10
24	16:38:45	15 s	14 s	Database schema: core entities and mapping decisions	2.6, V6
25	16:39:00	110 s	86 s	Testing on the Agile Testing Quadrants	3.1, 3.3
26	16:40:50	70 s	69 s	Scripted performance test: tool, scenario, target	3.2

Total slots 250 s; estimated speech 224 s. I come in after Lim (slide 21) and hand over to Pan (slide 27).

Slide 22 · Design problem: listing rules that depend on its state

16:37:50–16:38:20 · slot 30 s · estimate 30 s (66 words) · rubric 2.5, G4

CUE IN Liu Jin, one problem in full.

SAY

I'm Liu Jin, with one design problem in full. An admin can publish, delist, revise or delete a listing, but each action is allowed only in some states. `click 1/2` A brute-force design, which we rejected, checks the status inside all four operations. `click 2/2` In the sequence, publish branches on the status. What varies is the state of the listing, so we compared State, Strategy and a transition table.

HAND OVER (Liu Jin continues) The design with State. next: slide 23, Liu Jin

CUES

- Diagram: src/diagrams/state-before.svg. It is the brute-force alternative that backend/docs/design/admin-listings.md L67–68 rejects ("repeated status switches in every caller"), drawn as RT 3.5 allows ("using brute force design if necessary"). It never existed as code; the rules inside the switches are the real ones (ProductStatus.java L12–74).
- Real operation names: Product.publish(), delist(), revise(details), requireDeletable(); client AdminListingAppServiceImpl; endpoint PUT /api/v1/admin/products/{productId}/status (AdminListingController).
- Cause for redesign: GoF State applicability, "operations have large, multipart conditional statements that depend on the object's state". None of the causes on SDP 05a p15 names State, so we do not claim one. What varies: SDP 05a p13, "States of an object".
- Authorship, if asked: admin-listings.md names Eric as implementer and Liu Jin as reviewer; the report owner is Liu Jin, decided 24 Oct (#463). Answer: "I own this design and its report section; the first version was built with Eric in PR #72."

Slide 23 · After: the State pattern, and why the team uses a table

16:38:20–16:38:45 · slot 25 s · estimate 25 s (54 words) · rubric 2.5, 2.4, G10

SAY

We chose State, because each state changes how all four operations behave. `click 1/2` Product delegates to ProductStatus. Each state overrides only the moves it allows; the rest return 409. `click 2/2` Each state answers publish itself, with no switch. A new state costs a constant, overrides and tests. Teams have one move, so they keep a table.

HAND OVER (Liu Jin continues) Where the data lives. next: slide 24, Liu Jin

CUES

- Diagram: src/diagrams/state-after.svg (crops A2, B); the after-sequence alt has one operand per kind of ConcreteState (override: DRAFT, OFF_SALE; inherited default: ON_SALE), chosen by dynamic dispatch, not by a switch; listing statechart DRAFT -publish-> ON_SALE -delist-> OFF_SALE -publish-> ON_SALE, delete only in DRAFT: backup B13.
- Enum constant bodies: ProductStatus DRAFT, ON_SALE and OFF_SALE all have bodies; javac compiles each to an anonymous subclass (ProductStatus$1 … $3), so it is one class per state, shared and stateless (GoF State implementation 2). Defaults throw BizException(PRODUCT_STATE_CONFLICT), HTTP 409.
- Why State here: each state changes four operations, not only which moves are legal. DRAFT allows price, currency and stock edits and deletion; ON_SALE and OFF_SALE lock them; publish and delist differ per state. Implementation decisions: Product is immutable, so a transition returns a new revision with version + 1, and updateListing(…, expectedVersion) returns 412 on a stale version (Optimistic Offline Lock, a persistence mechanism, not GoF). Liability beyond the classes per state: AdminListingAppServiceImpl.changeStatus still switches on the target to pick publish or delist, and Product.isOnSale() compares the status for the browse filter.
- Why not State for the team: DealStatus has one move (OPEN → terminal) and its effects are events (slide 19), so a transition table is enough; State would add three classes with no behaviour (SDP 05a p35). Slide 4 records this change: the proposal promised State for the team lifecycle.
- Tests (G10): ProductStatusTest.onSalePublishThrowsStateConflict, offSaleDelistThrowsStateConflict, currencyOnly ChangeIsRejectedAfterPublication; ProductLifecycleTest (7). Report §3.5.5 (Liu Jin).

Slide 24 · Database schema: core entities and mapping decisions

16:38:45–16:39:00 · slot 15 s · estimate 14 s (31 words) · rubric 2.6, V6

SAY

The core schema. `click 1/2` Keys are UUIDv7, and unique indexes stop races: one membership per buyer per team, `click 2/2` one refund per payment. The application checks the links; we use no foreign keys.

HAND OVER Next, how we test it. next: slide 25, Liu Jin

CUES

- Diagram: src/diagrams/core-erd.svg: biz_product, biz_group_deal, biz_group_member, biz_purchase_intent, biz_payment, biz_order, biz_refund and sys_user, from Flyway V1–V28 (backend/blocbuy-bootstrap/src/main/resources/db/migration). Crops 2 (deal, membership: uq_biz_group_member_deal_user) and 6 (payment, refund: biz_refund_payment_id_key, an inline UNIQUE on payment_id).
- No FK or CHECK constraints is the team's DDL rule (.claude/skills/pgsql-ddl, V5 header), not a course rule; the application keeps integrity (QA: orphan rows). SAD 04a p55 maps associations to FK columns; we keep those columns without the constraint.
- Other details if asked: biz_purchase_intent has no status column (state from fulfilled_payment_id and expired_at); biz_group_deal.version is incremented, not compared; uq_biz_payment_request_key carries the Checkout idempotency key. All 29 tables: backup B4; zoomable copy: open src/diagrams/full-erd.svg in a browser tab.

Slide 25 · Testing on the Agile Testing Quadrants

16:39:00–16:40:50 · slot 110 s · estimate 86 s (186 words) · rubric 3.1, 3.3

SAY

We sort our tests into the Agile Testing Quadrants. `click 1/4` In quadrant one, most tests are unit tests in JUnit and Vitest. Integration tests run on real PostgreSQL, Redis and ElasticMQ in Testcontainers, and ArchUnit checks our architecture rules. These tests cover the complex classes the proposal named: the team state machine; the last seat, where twenty paid buyers race and only one gets it; and refunds, where a retry after a lost Stripe response reuses the same key. Two failing tests reproduced a join-quote bug before we fixed it. `click 2/4` Quadrant two tests the user stories: acceptance tests through real HTTP and signed webhooks, and eight Playwright specs on the preview build and on dev. The dev run caught a 502 while the host restarted. `click 3/4` Quadrant three is manual: a Stripe Sandbox run and cookie checks on dev. They found no defects, and no outside users tested the system. `click 4/4` Quadrant four uses tools: the load test on the next

slide, and scans of code, dependencies, images, infrastructure and secrets. Since this week, a passive ZAP scan also checks dev. Pan shows where the scans gate the pipeline.

IF OVER TIME If over time, drop "Two failing tests reproduced a join-quote bug before we fixed it." (it stays on the slide).

HAND OVER Next, the test for the performance target. next: slide 26, Liu Jin

CUES

- Model: EPAT 03 Test Driven Development v2.1 p7 (Agile Testing Quadrants: Q1 unit and component, automated; Q2 functional and story tests, automated and manual; Q3 exploratory, usability, UAT, manual; Q4 performance, load, security, tools) and p5 (levels: unit, integration, system). The level tag on each row is our classification. Placement: repository, SQL and contract integration tests are technology-facing, so Q1 at integration level; Q2 holds only story-level tests that exercise a user story end to end: StripeRolloutAcceptanceIT (8 tests: real HTTP endpoints, scheduler, Flyway schema, webhook signatures; only OAuth and Stripe network calls substituted) and the Playwright e2e specs.
- Counted in the repository on 22 Oct at a1c6ac6e: find backend -name "*Test.java" = 186 (185 + ArchitectureTest), 1,329 @Test/@ParameterizedTest; of the 185, 23 boot @SpringBootTest or Testcontainers (grep -l "@SpringBootTest|Testcontainers|@Container|PostgreSQLContainer", 22 Oct at 68325744), so 162 are unit level; "*IT.java" = 49, 45 of them start Testcontainers (postgres:18, redis:8.10, ElasticMQ), 242 annotations; ArchitectureTest.java = 17 rules (grep -c @ArchTest prints 18 because a comment matches); frontend/src "*.test.ts*" = 77; frontend/e2e = 8 specs, 24 test() declarations. Per module: backup B7.
- Complex classes (proposal §6.3 "unit tests concentrated on classes with complex behaviour"): state machine DealStatusTest, PaidGroupDealTest (terminalStatesRejectJoins, lastPaidMemberSucceedsWithoutMutatingOriginalSnapshot); capacity StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist (Spring Boot with production ports, PostgreSQL in Testcontainers; twenty paid JOINs released together; asserts 2 members, the leader and one joiner, and 19 refund obligations; a *Test by name, integration by level). StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat is not cited: it races through a test fake of the join path; refund idempotency RefundServiceTest.lostResponseRetriesSameDurableKey.
- Caught: (Q1) sprint-03/log/2026-10-07.md, two zero/negative-count red tests reproduced the quote defect (GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted; #422, fixed in #424; guard-order and golden-hash tests pin the refactoring: closedAndFullTeamsAreRejectedBeforeDuplicateMembership, admissionGuardsRejectMissingFactsAndPreservePaidRejectionPriority, historicalStartAndJoinFingerprintsRemainByteCompatible); ArchUnit controllers_are_validated required @Validated on InternalUserController (sprint-04/log/2026-10-13.md, #468). (Q2) ci-frontend runs 37102432667 (3 Oct) and 37182996807 (4 Oct): e2e-dev got 502 from /api/actuator/health inside deploy-existing-ec2 of the same commit. (Q3) none logged from the 20 Sep Sandbox run or the 8 Oct cookie check. (Q4) security.yml weekly full-history gitleaks: 24 matches, 20 SHA-256 digests in a contract baseline re-hashed and fingerprinted (PR #580), 4 badge-token findings still open (slide 32; docs/quality/quality-gates.md).
- Where it runs: ci-backend.yml build-quality (mvnw verify: unit, ArchUnit, ITs; required), contract-check (export, git diff --exit-code contracts/, openapi-diff, not required); ci-frontend.yml frontend-check (Vitest; required), e2e-preview (PRs), e2e-dev (after deploy-s3-dev). Latest local full backend verify: 2,048 passed, 1 opt-in performance skip (sprint-04/log/2026-10-21.md, base 30b75aaa).
- Q3 evidence: backend/docs/payment/stripe-sandbox-acceptance-20260920.md (browser START and JOIN through Hosted Checkout, local, 20 Sep); sprint-03 8 Oct log: Google sign-in on dev, Chrome DevTools shows blocbuy-token Secure and HttpOnly; hand mutation probes 12–13 Oct logs. No user acceptance testing with outside users; say so if asked.
- Q4 security: types only here (rubric 3.3). Tools and gates are on slide 30: SAST SonarCloud + SpotBugs, dependencies Trivy fs + Dependabot, image Trivy, IaC configuration Trivy config, secrets gitleaks. DAST: dast-baseline.yml (#607, merged 22 Oct), a passive OWASP ZAP baseline of selected anonymous dev responses, weekly on Monday, on demand and on PRs that change the scan itself (not on ordinary application PRs); findings advisory, scanner or coverage failures fail that job; no login, payment or active testing. Slide 25's Q4 detail and slide 30 list it, with its first findings (0 High, 2 Medium, 4 Low; #610). If the Mon 26 Oct 03:41 UTC scheduled dast-baseline run is still red after #682 (slide 30 cue), do not say the weekly scan works: say it is scheduled weekly and its last run was incomplete.

Slide 26 · Scripted performance test: tool, scenario, target

16:40:50–16:42:00 · slot 70 s · estimate 69 s (150 words) · rubric 3.2

SAY

Our performance test is a k6 script. It runs on the SWE5001 ECS environment, with an image built from dev on 6 October. We never load the shared dev host, because its database runs on the same machine. The load rises from 10 to 200 requests a second, one minute per step. Each step must keep p95 under 500 milliseconds and errors under 1

percent, and must not drop requests. The proposal asks for 2 seconds at 100 concurrent users. By Little's law, 100 users who wait half a second between clicks send about 190 requests a second. Our top step covers that load, and our limit is four times stricter. For team expiry, the sweep starts a minute after the previous run ends, so a team closes at most a minute plus the sweep time after its deadline. Locally, the worst case was 58 seconds. Yorik shows the result.

IF OVER TIME If over time, drop "We never load the shared dev host, because its database runs on the same machine." (Q&A covers it).

HAND OVER Over to Pan, who shows where these tests run in the pipeline. next: slide 27, Pan

CUES

- Tool and scenario: perf/k6/README.md §V2 and perf/k6/browse.js: grafana/k6:1.6.1 pinned by digest; ramping-arrival executor, 10, 20, 40, 60, 80, 100, 125, 150, 175, 200 iterations/s, 60 s each; one request per iteration, 7 list : 3 detail; no sleep; 100 preallocated VUs, 400 max; 10 s request timeout; gates per plateau p95 < 500 ms, error rate < 1%, zero dropped iterations (calibrate.mjs). Anonymous browse only: authenticated load needs real sessions.
- Environment: SWE5001 ECS, CloudFront d3lvq02o2e7tcn.cloudfront.net → ALB → ECS Fargate; the README says "Do not load-test shared SWE5006 dev". Say "SWE5001 environment, an image built from dev on 6 Oct" every time (image 6c7a99ab, 38 backend commits before 9c123c12); never call it a dev result.
- Target: proposal §6.3 "Interactive operations respond within 2 seconds under demonstration load (up to 100 concurrent users); team expiry is resolved within 1 minute of window lapse." Little's law N = X(R + Z): N = 100, Z = 0.5 s think, R ≈ 0.03 s gives X ≈ 189 requests/s. The 0.5 s think time is our assumption; with longer think times the offered load is lower.
- History, not the 5.3 result: SWE5001 V2 baseline 16 Oct (docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md), one monolith task, two runs of 57,600 requests, zero errors, every plateau p95 ≤ 57 ms; saturation not observed, so 200 requests/s is a tested lower bound. Local Node workload 30 Sep (docs/perf/sprint3-baseline.md): 100 closed-loop VUs, 200 ms think, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms; 30 expiries, worst observation lag 58.18 s. Local only.
- Expiry on ECS: the V4 burst (2,000 teams, perf/k6/SCALING.md) has not been run, so expiry evidence is local only. TeamExpirySweepJob runs with fixedDelay PT1M, so the bound is one interval plus the sweep time; measured worst lag 58.18 s with 30 teams (docs/perf/sprint3-baseline.md). The 5.3 run is the 16 Oct V2 repeat (run 20261016-v2-monolith-repeat, docs/swe5001/evidence/20261016-http-baseline-dashboard/v2-repeat-summary.json; #670); if a 24 Oct pre-run passes, swap in its numbers.

Questions I answer

Answer in 30–45 s: decision, reason, evidence or limitation. "A → B" means A starts and B adds.

Question	Answer	Who
The proposal promised JWT. Where is that change recorded?	In the backend skeleton plan of 30 Aug (decisions D6 and D7: a Sa-Token session in Redis, roles from SQL tables) and in frontend ADR 0002 (same-origin /api, HttpOnly cookie). We chose a server session because we can revoke it and the token never reaches JavaScript. There is no numbered backend ADR for it yet (open item 7).	Lim → Liu Jin
Is ProductStatus really the State pattern, or an enum switch?	It is State. Each enum constant with a body compiles to its own subclass, overrides only the moves it allows and returns the next state; the default methods refuse with 409. Product calls its state and does not branch on it. Evidence: ProductStatus.java, ProductStatusTest. One switch remains: AdminListingAppServiceImpl.changeStatus picks publish or delist from the requested target. The team uses a transition table (DealStatus.canTransitionTo), which we do not call State.	Liu Jin
Why not the State pattern for the team, as the proposal said?	Only OPEN has behaviour. SUCCEEDED and EXPIRED are final and refuse every move with the same error. A transition table (DealStatus.canTransitionTo) and the guards in GroupDeal cover that; State would add three classes with no behaviour. Listings differ: each state changes how four operations behave (slide 23). Slide 4 records the change.	Liu Jin → Eric
Is there a database per service?	On dev, no: one PostgreSQL database holds every table in schema domain. On ECS, yes: one RDS instance with one database per service. Each database applies the full Flyway history, and each service role is granted only the tables it owns (ADR 0008). Full list: B4.	Liu Jin
Without foreign keys, what stops orphan rows?	The application. Our DDL rule bans foreign keys and CHECK constraints; that is a team rule, not a course rule. Business tables carry a deleted flag, so a delete hides a row instead of breaking its links. Before a listing is deleted, the use case checks every reference: only a draft with no deals, quotes or price policies can go (AdminListingAppServiceImpl.delete, PRODUCT_REFERENCED). Unique indexes stop duplicate rows under races.	Liu Jin
How do you test payments without calling Stripe?	Stripe sits behind two ports, the Checkout gateway and the Refund gateway. Unit tests replace the port. StripeRolloutAcceptanceIT (8 tests) runs the real HTTP endpoints, the scheduler, the Flyway schema and the webhook signature checks; only OAuth and the calls to Stripe are replaced. On dev on 23 Oct, a paid Sandbox START settled and a natural expiry refunded once (PR #664, merged 24 Oct, bef39b72); a second buyer's paid JOIN was shown on dev on 24 Oct (system demo backup recording, #657).	Liu Jin → Yorik
Why Testcontainers rather than H2?	Our rules live in the database: unique keys, ON CONFLICT DO NOTHING, guarded updates and role grants. H2 accepts different SQL, so we test on real PostgreSQL 18, Redis 8.10 and ElasticMQ in containers. Example: OrderRepositoryIT.duplicatePaymentIs RejectedAtTheDatabase. The cost is Docker and time: the build-quality job takes about 13 minutes (B8).	Liu Jin
Do you do TDD?	For some changes. Four day logs record the red step: new PostgreSQL tests for the offer query failed first (27 Sep, #205); four admin-console changes started from failing tests (1 Oct, PRs #326–#329); the rate-limit tests failed first, 429 expected, 200 returned (3 Oct); two failing tests reproduced the join-quote bug before the fix (7 Oct). Commits are squashed, so the red step lives in the day logs (docs/agile/practices.md). Other changes add their	Liu Jin

Question	Answer	Who
	tests in the same pull request, and the 80% new-code gate requires them.	
What is your test coverage?	SonarCloud on dev, read 24 Oct (analyses of bef39b72, the last code change on dev): backend 99.4% and frontend 96.1% overall, both gates passed; on new code 99.3% and 96.2%. The gate needs at least 80% on new code (NFR-06). The frontend figure excludes the generated API client, the route tree and the app entry files (ci-frontend.yml, sonar.coverage.exclusions). Coverage does not measure assertion quality, and we run no mutation testing.	Liu Jin
How do you test the frontend against the API?	Contract first. The backend regenerates contracts/openapi.json, and CI fails if the committed file is out of date; openapi-diff fails a breaking change unless it is labelled. The frontend generates its client and its MSW handlers from that file (frontend ADRs 0003, 0011), and an unhandled request fails a test. The stubs prove only the frontend side.	Liu Jin → Lim
Why is the performance test not on dev?	Dev is one host, and its database runs on the same machine, so a load test would slow down the demo system and its database; the k6 README forbids it. The test runs on the SWE5001 ECS environment, with an image built from dev on 6 October (slide 26; the 16 Oct run is on slide 36). The local Node workload ran on 30 Sep (docs/perf/sprint3-baseline.md). Neither result is from dev, and each is labelled.	Liu Jin
Does CloudFront cache the API, so k6 measures the cache?	No. The /api/* behaviour is uncached, and the k6 preflight checks that before a run (perf/k6/README.md). Every request reaches the ALB and the ECS service.	Liu Jin
Is 200 requests a second the same as 100 concurrent users?	Not directly, so we convert. k6 sends requests at a set rate; the proposal counts users. By Little's law, N = X(R + Z): 100 users who wait 0.5 s and get answers in about 30 ms send about 190 requests a second, below our 200. Our limit is p95 under 500 ms, four times stricter than 2 s. The 0.5 s wait is our assumption. The local run on 30 Sep used exactly 100 users with 200 ms waits, and every p95 was 57 ms or less.	Liu Jin
Is the last-seat race test a unit test?	No, it is an integration test. StripeLocalIntegrationTest.twenty BuyersRaceForLastSeatAndNineteenRefundObligationsPersist sends twenty paid buyers through the production intent and fulfilment ports against PostgreSQL 18 in Testcontainers. StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTake LastSeat also uses real PostgreSQL, but its provider, group and order parts are test doubles (class javadoc), so we do not cite it. The unit tests of the same rule are PaidGroupDealTest (the aggregate guards) and DealStatusTest (the transition table).	Liu Jin
Why no end-user acceptance testing?	Quadrant three is manual checks by the team, and they are logged: the Stripe Sandbox browser run (20 Sep, local) and the sign-in cookie check on dev (8 Oct). We had no outside users in the course, so we do not claim UAT.	Liu Jin
Is 401 vs 403 vs 400 deliberate?	Yes. The Sa-Token interceptor checks login on every /api/v1 path except anonymous GETs of listings and invites, so no session is 401 COMMON_401_001; a signed-in buyer on an admin endpoint fails StpUtil.checkRole(ADMIN) and gets 403 COMMON_403_001; a money write without the X-BlocBuy-Request marker is 400 COMMON_400_001, and a wrong value is 403 (StripeCheckoutController.checkCsrf). Mapping: GlobalExceptionHandler.	Yorik → Liu Jin

My open items before the 25 Oct rehearsal

#	Item	Owner	Slide
7	No numbered backend ADR for: the auth change from JWT to the Sa-Token session (open decision 6); MyBatis instead of the proposal's JPA; the scheduling design; the concurrency approach (open decision 3); the team lifecycle as a transition table instead of the proposal's State; the DevSecOps changes from proposal §5.2 (OWASP Dependency-Check replaced by Trivy deps, Trivy IaC and deps not required checks, three OIDC roles split per job instead of plan/deploy/apply, two IAM users with one access key against "no static credentials"). Not writable before 26 Oct: state the gap in report Appendix B (ADR index and the decisions without an ADR, #681), and write ADRs only if the team chooses	Liu Jin, Eric, Pan	4, 10
17	Recount the test numbers for slide 25 on 25 Oct	Liu Jin	25

On the day

- IDE open on ProductStatus, Product, ProductStatusTest and ProductLifecycleTest (rubric G10).
- Zoomable full ERD (src/diagrams/full-erd.svg) open in a browser tab for slide 24.